```
import { timingSafeEqual } from 'node:crypto';
import express, { Router } from 'express';
import pg from 'pg';
import { Deduplicator } from './dedup.js';
import { errorInfo, logger, maskPhone } from '../utils/logger.js';

/**
 * Cobrança pelo Asaas + controle de acesso pelo número do WhatsApp.
 *
 * - Quem pagou: acesso liberado por ACCESS_DAYS a partir do pagamento.
 * - Quem não pagou: tem FREE_ANALYSES análises grátis; depois recebe o link de p
 * - O próprio Asaas guarda quem pagou (cliente com externalReference = número),
 *   reiniciar o servidor não perde os assinantes.
 */

export type Access = 'paid' | 'free' | 'blocked';

export interface Paywall {
  check(phone: string): Promise<Access>;
  /** Marca uma análise grátis como usada. */
  useFree(phone: string): void;
  /** Mensagem com o link de pagamento para esse número. */
  payMessage(phone: string): Promise<string>;
  /** Trata o webhook do Asaas. Devolve o número liberado (se houver). */
  onWebhook(body: unknown): Promise<string | null>;
}

export interface PaywallOptions {
  apiKey: string;
  sandbox: boolean;
  planName: string;
  price: number;
  accessDays: number;
  freeAnalyses: number;
  /** Números que nunca pagam (dono, influenciador). */
  exempt: string[];
  /** Para onde o cliente volta depois de pagar. */
  siteUrl: string;
  fetchImpl?: typeof fetch;
  now?: () => number;
}

const DAY = 86_400_000;
```

```
const digits = (s: string) => s.replace(/\D/g, '');
const brl = (v: number) => v.toFixed(2).replace('.', ',');
const isObj = (v: unknown): v is Record<string, unknown> => typeof v === 'object'

/** Formas do mesmo número (com e sem o 9° dígito brasileiro). */
function variants(phone: string): string[] {
  const d = digits(phone);
  const out = new Set([d]);
  if (d.startsWith('55') && d.length === 13 && d[4] === '9') out.add(d.slice(0, 4
  if (d.startsWith('55') && d.length === 12) out.add(d.slice(0, 4) + '9' + d.slic
  return [...out];
}

export function createPaywall(opts: PaywallOptions): Paywall {
  const doFetch = opts.fetchImpl ?? fetch;
  const now = opts.now ?? Date.now;
  const base = opts.sandbox ? 'https://api-sandbox.asaas.com/v3' : 'https://api.a
  const checkoutBase = opts.sandbox ? 'https://sandbox.asaas.com' : 'https://asaa
  const exempt = new Set(opts.exempt.flatMap(variants));

  const paidUntil = new Map<string, number>(); // cache: número -> fim do acesso
  const negative = new Map<string, number>(); // cache curto de "não pagou"
  const freeUsed = new Map<string, number>();
  const expired = new Set<string>(); // já pagou antes, mas venceu
  const checkouts = new Map<string, string>(); // checkoutId -> número
  const openLink = new Map<string, { url: string; until: number }>();

  async function api(path: string, init: RequestInit = {}): Promise<Record<string
    const res = await doFetch(base + path, {
      ...init,
      headers: { access_token: opts.apiKey, 'Content-Type': 'application/json', '
      signal: AbortSignal.timeout(15_000),
    });
    const text = await res.text();
    let data: unknown = {};
    try {
      data = text ? JSON.parse(text) : {};
    } catch {
      /* corpo não-JSON */
    }
    if (!res.ok) throw new Error(`Asaas ${init.method ?? 'GET'} ${path.split('?')
    return isObj(data) ? data : {};
  }

  const list = (d: Record<string, unknown>) => (Array.isArray(d.data) ? d.data.fi
```

```
/** Consulta no Asaas até quando esse número tem acesso pago. 0 = não pagou. */
async function lookupPaidUntil(phone: string): Promise<number> {
  let best = 0;
  for (const v of variants(phone)) {
    const customers = list(await api(`/customers?externalReference=${encodeURIC
    for (const c of customers) {
      if (typeof c.id !== 'string') continue;
      const payments = list(await api(`/payments?customer=${encodeURIComponent(
      for (const p of payments) {
        if (p.status !== 'RECEIVED' && p.status !== 'CONFIRMED' && p.status !==
        const when = Date.parse(String(p.paymentDate ?? p.confirmedDate ?? p.cl
        if (Number.isFinite(when)) best = Math.max(best, when + opts.accessDays
      }
    }
  }
  return best;
}

function grant(phone: string, until: number) {
  for (const v of variants(phone)) {
    paidUntil.set(v, Math.max(paidUntil.get(v) ?? 0, until));
    negative.delete(v);
    openLink.delete(v);
    expired.delete(v);
  }
}

async function createCheckout(phone: string): Promise<string> {
  const key = digits(phone);
  const cached = openLink.get(key);
  if (cached && cached.until > now()) return cached.url;
  const site = opts.siteUrl.replace(/\/$/, '');
  const data = await api('/checkouts', {
    method: 'POST',
    body: JSON.stringify({
      billingTypes: ['PIX', 'CREDIT_CARD'],
      chargeTypes: ['DETACHED'],
      minutesToExpire: 1440,
      externalReference: key,
      callback: { successUrl: `${site}/?pago=1`, cancelUrl: `${site}/`, expired
      items: [{ name: opts.planName, description: `${opts.accessDays} dias de a
    }),
  });
  const id = typeof data.id === 'string' ? data.id : '';
  if (!id) throw new Error('Asaas não devolveu o id do checkout.');
  checkouts.set(id, key);
```

```
  const url = typeof data.link === 'string' && data.link.startsWith('https://')
  openLink.set(key, { url, until: now() + 23 * 3600_000 });
  logger.info('Link de pagamento criado', { to: maskPhone(key) });
  return url;
}

return {
  async check(phone) {
    const vs = variants(phone);
    if (vs.some((v) => exempt.has(v))) return 'paid';
    const t = now();
    if (vs.some((v) => (paidUntil.get(v) ?? 0) > t)) return 'paid';
    const neg = Math.max(...vs.map((v) => negative.get(v) ?? 0));
    if (neg <= t) {
      try {
        const until = await lookupPaidUntil(phone);
        if (until > t) {
          grant(phone, until);
          return 'paid';
        }
        for (const v of vs) {
          negative.set(v, t + 60_000);
          if (until > 0) expired.add(v);
        }
      } catch (err) {
        logger.error('Falha ao consultar pagamento no Asaas', errorInfo(err));
      }
    }
    const used = Math.max(...vs.map((v) => freeUsed.get(v) ?? 0));
    return used < opts.freeAnalyses ? 'free' : 'blocked';
  },

  useFree(phone) {
    for (const v of variants(phone)) freeUsed.set(v, (freeUsed.get(v) ?? 0) + 1
  },

  async payMessage(phone) {
    const vs = variants(phone);
    const reason = vs.some((v) => expired.has(v)) ? 'expired' : vs.some((v) =>
    const head =
      reason === 'used-free'
        ? '🎱 Essa foi sua análise grátis! Curtiu? Pra continuar recebendo as j
        : reason === 'expired'
          ? '⏰ Seu acesso venceu. Pra continuar recebendo as jogadas, renova a
          : '🎱 Pra receber as jogadas, libera seu acesso:';
    try {
```

```
      const url = await createCheckout(phone);
      return `${head}\n\n👉 ${url}\n\n${opts.planName}: R$ ${brl(opts.price)} p
    } catch (err) {
      logger.error('Falha ao criar link de pagamento', errorInfo(err));
      return `${head}\n\nTô com problema pra gerar o link agora. Tenta de novo
    }
  },

  async onWebhook(body) {
    if (!isObj(body)) return null;
    const event = typeof body.event === 'string' ? body.event : '';
    logger.info('Webhook do Asaas', { event });
    if (!['CHECKOUT_PAID', 'PAYMENT_RECEIVED', 'PAYMENT_CONFIRMED', 'PAYMENT_RE

    let phone: string | undefined;
    let customer: string | undefined;
    if (isObj(body.checkout)) {
      const co = body.checkout;
      phone = (typeof co.id === 'string' && checkouts.get(co.id)) || (typeof co
      customer = typeof co.customer === 'string' ? co.customer : undefined;
    }
    if (isObj(body.payment)) {
      const p = body.payment;
      customer ??= typeof p.customer === 'string' ? p.customer : undefined;
      if (!phone && typeof p.checkoutSession === 'string') phone = checkouts.ge
      if (!phone && typeof p.externalReference === 'string' && /^\d{10,15}$/.te
    }
    if (!phone && customer) {
      try {
        const c = await api(`/customers/${encodeURIComponent(customer)}`);
        if (typeof c.externalReference === 'string' && /^\d{10,15}$/.test(c.ext
      } catch (err) {
        logger.warn('Não consegui ler o cliente do Asaas', errorInfo(err));
      }
    }
    if (!phone) {
      logger.warn('Pagamento sem número associado', { event, keys: Object.keys(
      return null;
    }
    // Grava o número no cliente do Asaas: é assim que o acesso sobrevive a rei
    if (customer) {
      try {
        await api(`/customers/${encodeURIComponent(customer)}`, { method: 'POST
      } catch (err) {
        logger.warn('Não consegui gravar o número no cliente do Asaas', errorIn
      }
```

```
      }
      const already = (paidUntil.get(phone) ?? 0) > now();
      grant(phone, now() + opts.accessDays * DAY);
      logger.info('Acesso liberado', { to: maskPhone(phone), event });
      return already ? null : phone;
    },
  };
}

export const MSG_PAID = '✅ Pagamento confirmado, acesso liberado! 🎱 Manda a fot
export const MSG_FREE_HINT =
  'Fala! 🎱 Sou o treinador de sinuca. Você tem 1 análise grátis: manda uma foto

// ================================================================================
// HOTMART
// ================================================================================

// ---------- Banco: quem pagou ----------
/**
 * Onde fica guardado quem pagou (Hotmart). Com DATABASE_URL usa Postgres e sobre
 * reinícios; sem ele, guarda só na memória (perde tudo quando o servidor reinici
 *
 * Tabelas próprias da Sinuca (prefixo sinuca_): não toca em nenhuma outra tabela
 */

export interface Sale {
  email?: string;
  subscriber?: string;
  transaction?: string;
}

export interface AccessStore {
  readonly persistent: boolean;
  init(): Promise<void>;
  /** Maior data de fim de acesso entre essas formas do número (0 = nunca pagou).
  paidUntil(phones: string[]): Promise<number>;
  grant(phone: string, until: number, sale: Sale): Promise<void>;
  /** Número já ligado a essa assinatura ou e-mail (renovações). */
  findPhone(sale: Sale): Promise<string | null>;
  /** Corta o acesso (reembolso/chargeback). Devolve os números afetados. */
  revoke(sale: Sale): Promise<string[]>;
  /** Venda aprovada sem número: fica esperando o cliente mandar o e-mail no What
  saveUnmatched(until: number, sale: Sale): Promise<void>;
  /** Pega uma venda sem número pelo e-mail. Devolve o fim do acesso, ou 0. */
  claimUnmatched(email: string, now: number): Promise<{ until: number; sale: Sale
  freeUsed(phones: string[]): Promise<number>;
```

```
  addFree(phone: string): Promise<void>;
}

const low = (s?: string) => (s ? s.trim().toLowerCase() : undefined);

export function createMemoryStore(): AccessStore {
  const access = new Map<string, { until: number } & Sale>();
  const unmatched: ({ until: number } & Sale)[] = [];
  const free = new Map<string, number>();
  return {
    persistent: false,
    async init() {},
    async paidUntil(phones) {
      return Math.max(0, ...phones.map((p) => access.get(p)?.until ?? 0));
    },
    async grant(phone, until, sale) {
      const cur = access.get(phone);
      access.set(phone, { ...cur, ...sale, email: low(sale.email) ?? cur?.email,
    },
    async findPhone(sale) {
      for (const [phone, a] of access) {
        if (sale.subscriber && a.subscriber === sale.subscriber) return phone;
      }
      for (const [phone, a] of access) {
        if (sale.email && a.email === low(sale.email)) return phone;
      }
      return null;
    },
    async revoke(sale) {
      const out: string[] = [];
      for (const [phone, a] of access) {
        if ((sale.transaction && a.transaction === sale.transaction) || (sale.sub
          a.until = 0;
          out.push(phone);
        }
      }
      for (let i = unmatched.length - 1; i >= 0; i--) {
        const u = unmatched[i];
        if ((sale.transaction && u.transaction === sale.transaction) || (sale.sub
      }
      return out;
    },
    async saveUnmatched(until, sale) {
      unmatched.push({ ...sale, email: low(sale.email), until });
    },
    async claimUnmatched(email, now) {
```

```
      const i = unmatched.findIndex((u) => u.email === low(email) && u.until > no
      if (i < 0) return null;
      const [u] = unmatched.splice(i, 1);
      return { until: u.until, sale: u };
    },
    async freeUsed(phones) {
      return Math.max(0, ...phones.map((p) => free.get(p) ?? 0));
    },
    async addFree(phone) {
      free.set(phone, (free.get(phone) ?? 0) + 1);
    },
  };
}

export function createPgStore(databaseUrl: string): AccessStore {
  const internal = !/\.render\.com|amazonaws|neon|supabase/.test(databaseUrl);
  const pool = new pg.Pool({
    connectionString: databaseUrl,
    max: 3,
    ssl: internal ? undefined : { rejectUnauthorized: false },
    connectionTimeoutMillis: 10_000,
  });
  pool.on('error', (err) => logger.error('Erro na conexão com o banco', errorInfo
  const q = (sql: string, params: unknown[] = []) => pool.query(sql, params);
  const ms = (v: unknown) => (v instanceof Date ? v.getTime() : 0);

  return {
    persistent: true,
    async init() {
      await q(`CREATE TABLE IF NOT EXISTS sinuca_access (
        phone text PRIMARY KEY,
        paid_until timestamptz NOT NULL,
        email text,
        subscriber text,
        transaction text,
        updated_at timestamptz NOT NULL DEFAULT now())`);
      await q(`CREATE TABLE IF NOT EXISTS sinuca_unmatched (
        id bigserial PRIMARY KEY,
        paid_until timestamptz NOT NULL,
        email text,
        subscriber text,
        transaction text,
        created_at timestamptz NOT NULL DEFAULT now())`);
      await q(`CREATE TABLE IF NOT EXISTS sinuca_free (phone text PRIMARY KEY, us
    },
    async paidUntil(phones) {
```

```
  const r = await q('SELECT max(paid_until) AS u FROM sinuca_access WHERE pho
  return ms(r.rows[0]?.u);
},
async grant(phone, until, sale) {
  await q(
    `INSERT INTO sinuca_access (phone, paid_until, email, subscriber, transac
     ON CONFLICT (phone) DO UPDATE SET
       paid_until = GREATEST(sinuca_access.paid_until, EXCLUDED.paid_until),
       email = COALESCE(EXCLUDED.email, sinuca_access.email),
       subscriber = COALESCE(EXCLUDED.subscriber, sinuca_access.subscriber),
       transaction = COALESCE(EXCLUDED.transaction, sinuca_access.transaction
       updated_at = now()`,
    [phone, until, low(sale.email) ?? null, sale.subscriber ?? null, sale.tra
  );
},
async findPhone(sale) {
  if (sale.subscriber) {
    const r = await q('SELECT phone FROM sinuca_access WHERE subscriber = $1
    if (r.rows[0]) return String(r.rows[0].phone);
  }
  if (sale.email) {
    const r = await q('SELECT phone FROM sinuca_access WHERE email = $1 ORDER
    if (r.rows[0]) return String(r.rows[0].phone);
  }
  return null;
},
async revoke(sale) {
  const cond = '(transaction = $1 OR subscriber = $2)';
  const args = [sale.transaction ?? '__none__', sale.subscriber ?? '__none__'
  const r = await q(`UPDATE sinuca_access SET paid_until = to_timestamp(0), u
  await q(`DELETE FROM sinuca_unmatched WHERE ${cond}`, args);
  return r.rows.map((x) => String(x.phone));
},
async saveUnmatched(until, sale) {
  await q('INSERT INTO sinuca_unmatched (paid_until, email, subscriber, trans
    until,
    low(sale.email) ?? null,
    sale.subscriber ?? null,
    sale.transaction ?? null,
  ]);
},
async claimUnmatched(email, now) {
  const r = await q(
    `DELETE FROM sinuca_unmatched WHERE id = (
       SELECT id FROM sinuca_unmatched WHERE email = $1 AND paid_until > to_t
     RETURNING paid_until, email, subscriber, transaction`,
```

```
        [low(email), now],
      );
      const row = r.rows[0];
      if (!row) return null;
      return { until: ms(row.paid_until), sale: { email: row.email ?? undefined,
    },
    async freeUsed(phones) {
      const r = await q('SELECT max(used) AS u FROM sinuca_free WHERE phone = ANY
      return Number(r.rows[0]?.u ?? 0);
    },
    async addFree(phone) {
      await q('INSERT INTO sinuca_free (phone, used) VALUES ($1, 1) ON CONFLICT (
    },
  };
}

// ---------- Paywall da Hotmart ----------
/**
 * Cobrança pela Hotmart + controle de acesso pelo número do WhatsApp.
 *
 * - O link de pagamento leva o número do cliente no parâmetro sck da Hotmart; o
 *   devolve esse sck e assim o bot sabe quem liberar.
 * - Se o sck não vier (comprou por outro link), tenta: mesma assinatura/e-mail j
 *   depois o telefone digitado no checkout. Se nada bater, guarda a venda e o cl
 *   mandando o e-mail da compra no WhatsApp.
 * - Renovação mensal (nova "Compra aprovada") soma mais ACCESS_DAYS. Reembolso/c
 */

export interface HotmartPaywallOptions {
  checkoutUrl: string;
  productId?: string;
  planName: string;
  price: number;
  accessDays: number;
  /** Dias extras de folga para a renovação cair sem travar o cliente. */
  graceDays: number;
  freeAnalyses: number;
  exempt: string[];
  store: AccessStore;
  now?: () => number;
}

const str = (v: unknown) => (typeof v === 'string' && v.trim() ? v.trim() : typeo
const EMAIL = /[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}/;

/** Telefone digitado no checkout → formato do WhatsApp (55 + DDD + número). */
```

```
function normalizeBr(raw: string | undefined): string | undefined {
  if (!raw) return undefined;
  let d = digits(raw);
  if (d.length === 10 || d.length === 11) d = '55' + d;
  return /^55\d{10,11}$/.test(d) ? d : undefined;
}

const PAY_EVENTS = new Set(['PURCHASE_APPROVED']);
const CUT_EVENTS = new Set(['PURCHASE_REFUNDED', 'PURCHASE_CHARGEBACK', 'PURCHASE

export type ClaimResult = 'ok' | 'notfound' | 'noemail';
export type HotmartPaywall = Paywall & { claim(phone: string, text: string): Prom

export function createHotmartPaywall(opts: HotmartPaywallOptions): HotmartPaywall
  const now = opts.now ?? Date.now;
  const store = opts.store;
  const exempt = new Set(opts.exempt.flatMap(variants));
  const paidCache = new Map<string, number>();
  const negative = new Map<string, number>();
  const freeLocal = new Map<string, number>();
  const expired = new Set<string>();

  function link(phone: string) {
    const u = new URL(opts.checkoutUrl);
    u.searchParams.set('sck', digits(phone));
    return u.toString();
  }

  function remember(phone: string, until: number) {
    for (const v of variants(phone)) {
      paidCache.set(v, Math.max(paidCache.get(v) ?? 0, until));
      negative.delete(v);
      expired.delete(v);
    }
  }
  function forget(phone: string) {
    for (const v of variants(phone)) {
      paidCache.delete(v);
      negative.delete(v);
    }
  }

  async function grant(phone: string, sale: Sale): Promise<boolean> {
    const vs = variants(phone);
    const before = Math.max(...vs.map((v) => paidCache.get(v) ?? 0));
    const until = now() + (opts.accessDays + opts.graceDays) * DAY;
```

```
  await store.grant(digits(phone), until, sale);
  remember(phone, until);
  return before <= now();
}

return {
  async check(phone): Promise<Access> {
    const vs = variants(phone);
    if (vs.some((v) => exempt.has(v))) return 'paid';
    const t = now();
    if (vs.some((v) => (paidCache.get(v) ?? 0) > t)) return 'paid';
    if (Math.max(...vs.map((v) => negative.get(v) ?? 0)) <= t) {
      try {
        const until = await store.paidUntil(vs);
        if (until > t) {
          remember(phone, until);
          return 'paid';
        }
        for (const v of vs) {
          negative.set(v, t + 30_000);
          if (until > 0) expired.add(v);
        }
      } catch (err) {
        logger.error('Falha ao consultar acesso no banco', errorInfo(err));
      }
    }
    let used = Math.max(...vs.map((v) => freeLocal.get(v) ?? 0));
    if (used < opts.freeAnalyses) {
      try {
        used = Math.max(used, await store.freeUsed(vs));
      } catch (err) {
        logger.warn('Falha ao ler análises grátis no banco', errorInfo(err));
      }
    }
    return used < opts.freeAnalyses ? 'free' : 'blocked';
  },

  useFree(phone) {
    for (const v of variants(phone)) freeLocal.set(v, (freeLocal.get(v) ?? 0) +
    void store.addFree(digits(phone)).catch((err) => logger.warn('Falha ao grav
  },

  async payMessage(phone) {
    const vs = variants(phone);
    const head = vs.some((v) => expired.has(v))
      ? '⏰ Seu acesso venceu. Pra continuar recebendo as jogadas, renova aqui:
```

```
    : vs.some((v) => (freeLocal.get(v) ?? 0) > 0)
      ? '🎱 Essa foi sua análise grátis! Curtiu? Pra continuar recebendo as j
      : '🎱 Pra receber as jogadas, libera seu acesso:';
  return (
    `${head}\n\n👉 ${link(phone)}\n\n` +
    `${opts.planName}: R$ ${brl(opts.price)} por mês, no Pix ou cartão. Assim
    `Já pagou e não liberou? Me manda aqui o *e-mail* que você usou na compra
  );
},

async onWebhook(body) {
  if (!isObj(body)) return null;
  const event = str(body.event) ?? '';
  const data = isObj(body.data) ? body.data : {};
  const product = isObj(data.product) ? data.product : {};
  const buyer = isObj(data.buyer) ? data.buyer : {};
  const purchase = isObj(data.purchase) ? data.purchase : {};
  const origin = isObj(purchase.origin) ? purchase.origin : {};
  const subscription = isObj(data.subscription) ? data.subscription : {};
  const subscriberObj = isObj(subscription.subscriber) ? subscription.subscri
  const productId = str(product.id);
  logger.info('Webhook da Hotmart', { event, productId, status: str(purchase.

  if (opts.productId && productId && productId !== opts.productId) {
    logger.info('Webhook de outro produto da Hotmart, ignorado', { productId
    return null;
  }

  const sale: Sale = {
    email: str(buyer.email) ?? str(subscriberObj.email),
    subscriber: str(subscriberObj.code),
    transaction: str(purchase.transaction),
  };

  if (CUT_EVENTS.has(event)) {
    const phones = await store.revoke(sale);
    phones.forEach(forget);
    logger.info('Acesso cortado (reembolso/chargeback)', { event, numeros: ph
    return null;
  }
  if (!PAY_EVENTS.has(event)) return null;

  const sck = digits(str(origin.sck) ?? '');
  let phone = /^\d{10,15}$/.test(sck) ? sck : undefined;
  let how = 'sck';
  if (!phone) {
```

```
        phone = (await store.findPhone(sale)) ?? undefined;
        how = 'assinatura/e-mail';
      }
      if (!phone) {
        phone = normalizeBr(str(buyer.checkout_phone) ?? str(buyer.phone));
        how = 'telefone do checkout';
      }
      if (!phone) {
        await store.saveUnmatched(now() + (opts.accessDays + opts.graceDays) * DA
        logger.warn('Compra aprovada sem número: aguardando o cliente mandar o e-
        return null;
      }
      const isNew = await grant(phone, sale);
      logger.info('Acesso liberado', { to: maskPhone(phone), how, recorrencia: st
      return isNew ? phone : null;
    },

    async claim(phone: string, text: string): Promise<ClaimResult> {
      const email = text.match(EMAIL)?.[0];
      if (!email) return 'noemail';
      const found = await store.claimUnmatched(email, now());
      if (!found) {
        const owner = await store.findPhone({ email });
        // Já liberado para este mesmo número (ex.: número com/sem 9).
        if (owner && variants(phone).includes(owner)) {
          const until = await store.paidUntil(variants(owner));
          if (until > now()) {
            remember(phone, until);
            return 'ok';
          }
        }
        logger.info('E-mail sem compra pendente', { to: maskPhone(phone) });
        return 'notfound';
      }
      await store.grant(digits(phone), found.until, found.sale);
      remember(phone, found.until);
      logger.info('Acesso liberado pelo e-mail da compra', { to: maskPhone(phone)
      return 'ok';
    },
  };
}

// ---------- Configuração (variáveis de ambiente) ----------
/** Configuração da cobrança pela Hotmart. Ativa quando HOTMART_CHECKOUT_URL esti
export interface HotmartConfig {
  hottok: string;
```

```
  checkoutUrl: string;
  productId: string | undefined;
  planName: string;
  price: number;
  accessDays: number;
  graceDays: number;
  freeAnalyses: number;
  exempt: string[];
  databaseUrl: string | undefined;
}

export function loadHotmartConfig(env: NodeJS.ProcessEnv = process.env): HotmartC
  const checkoutUrl = env.HOTMART_CHECKOUT_URL?.trim();
  if (!checkoutUrl) return null;
  const problems: string[] = [];
  if (!/^https:\/\/\S+$/.test(checkoutUrl)) problems.push('HOTMART_CHECKOUT_URL c
  const hottok = env.HOTMART_HOTTOK?.trim() ?? '';
  if (!hottok) problems.push('HOTMART_HOTTOK');
  const n = (name: string, def: number) => {
    const raw = env[name]?.trim().replace(',', '.');
    const v = raw ? Number(raw) : def;
    if (!Number.isFinite(v) || v < 0) problems.push(`${name} numérico`);
    return v;
  };
  const cfg: HotmartConfig = {
    hottok,
    checkoutUrl,
    productId: env.HOTMART_PRODUCT_ID?.trim() || undefined,
    planName: env.PLAN_NAME?.trim() || 'Sinuca AI Mensal',
    price: n('PLAN_PRICE', 29.9),
    accessDays: n('ACCESS_DAYS', 30),
    graceDays: n('GRACE_DAYS', 3),
    freeAnalyses: n('FREE_ANALYSES', 1),
    exempt: (env.FREE_NUMBERS ?? '').split(',').map((x) => x.replace(/\D/g, '')).
    databaseUrl: env.DATABASE_URL?.trim() || undefined,
  };
  if (problems.length) throw new Error(`Hotmart: faltando/errado: ${problems.join
  return cfg;
}

// ---------- Webhook /hotmart/webhook ----------
export interface HotmartWebhookOptions {
  /** Hottok da conta Hotmart (vem no header X-HOTMART-HOTTOK ou no corpo, campo
  hottok: string;
  paywall: Paywall;
  notify: (phone: string, text: string) => Promise<void>;
```

```
  dedup?: Deduplicator;
}

function same(a: string, b: string) {
  const x = Buffer.from(a);
  const y = Buffer.from(b);
  return x.length === y.length && timingSafeEqual(x, y);
}

export function createHotmartRouter(opts: HotmartWebhookOptions): Router {
  const router = Router();
  const dedup = opts.dedup ?? new Deduplicator();

  router.post('/hotmart/webhook', express.json({ limit: '1mb' }), (req, res) => {
    const body = (req.body ?? {}) as { id?: unknown; hottok?: unknown };
    const given = String(req.get('x-hotmart-hottok') ?? (typeof body.hottok === '
    if (!same(given, opts.hottok)) {
      logger.warn('Webhook da Hotmart com hottok inválido');
      res.sendStatus(401);
      return;
    }
    res.sendStatus(200);

    if (typeof body.id === 'string' && !dedup.firstTime(`hotmart:${body.id}`)) re

    void opts.paywall
      .onWebhook(req.body)
      .then(async (phone) => {
        if (!phone) return;
        try {
          await opts.notify(phone, MSG_PAID);
        } catch (err) {
          logger.error('Pago, mas não consegui avisar no WhatsApp', { to: maskPho
        }
      })
      .catch((err) => logger.error('Erro no webhook da Hotmart', errorInfo(err)))
  });

  return router;
}
```